Софийски университет „Климент Охридски“
Факултет по математика и информатика

ПРОЕКТ
по МРЕЖОВО ПРОГРАМИРАНЕ

летен семестър 2024/2025

АЛГОРИТЪМ НА ЙЕН

Изготвил: Стоян Стоянов Иванов
Специалност: Информатика, 3 курс
Факултетен номер: 9MI0400132

Ръководител на курса:
доц. Стела Русева

май 2025 г.

СЪДЪРЖАНИЕ

1. Обща цел

2. Модули и структури
- 2.1. Граф
- 2.2. Алгоритми
- 2.3. Мрежова комуникация

3. Описание на алгоритъма на Йен
- 3.1. Основни дефиниции
- 3.2. Стъпка по стъпка
- 3.3. Сложност

4. Изграждане и използване
- 4.1. Изграждане
- 4.2. Използване

5. Примерни резултати
- 5.1 Алгоритъм на Дейкстра
- 5.2. Алгоритъм на Йен
- 5.3. Потребителски зададени входни данни

6. Заключение
- 6.1. Обобщение
- 6.2. Използвана литература

1. Обща цел

Целта на настоящия проект е да се реализира софтуерна система, която намира k най-кратки пътища между два върха в насочен и претеглен граф чрез алгоритъма на Йен (Yen's algorithm). Тази система се състои от клиент-сървърна архитектура, в която клиентът предоставя входните данни - броя на ребрата, началния и крайния връх, както и броя на търсените пътища, а сървърът извършва изчисленията и връща резултатите. Алгоритъмът на Йен по същество е разширение на алгоритъма на Дейкстра. Идеята на цялата система е не само да изчисли коректно и ефективно търсените пътища, но и да демонстрира реална употреба на алгоритмични подходи в мрежов анализ чрез ясна, разширяема и добре структурирана Java архитектура.

2. Модули и структури

Проектът е реализиран на езика Java с ясна модулна структура, която позволява лесна поддръжка, тестване и бъдещо разширяване. Всеки основен компонент на системата е капсулиран в самостоятелен модул, като спазва Single Responsibility Principle и разделение на логиката.

2.1. Граф

Съдържа основните структури за представяне на графа, върховете и пътищата:

- Vertex – Представлява връх в графа. Използва record за ефективност и неизменимост. Всеки връх има: label – човекоразбираемо име, което може да съвпада с ID, и id – уникален идентификатор;
- Edge – Представлява ребро между два върха с определена тежест. Всеки обект съдържа: source – начален връх, destination – краен връх, и weight – тежест на реброто. Методът compareBySourceAndDestination() се използва за сравнение на ребра при филтрации и изключване;
- Path – Представлява път в графа. Всеки път се състои от: списък от върхове, през които преминава пътят, и weight – общата дължина на пътя. Методите са getEdges() – генерира списък от ребра, съответстващи на пътя, join() – съединява два пътя, и equals() – сравнява пътища по върховете;
- Graph – Представлява цялостния граф чрез списък от ребра. Всеки обект съдържа списък от всички ребра в графа.

2.2. Алгоритми

Съдържа имплементации на използваните алгоритми за търсене на пътища.

- Dijkstra – Клас, реализиращ класическия алгоритъм на Дейкстра с възможност за изключване на ребра. Поддържа намиране на най-кратък път от

източник до цел, пресмятане на разстояния до всички достижими върхове и филтриране на определени ребра чрез excludedEdges;

- Yen – Клас, реализиращ алгоритъма на Йен за намиране на k най-кратки пътища. Използва многократно алгоритъма на Дейкстра като подпроблем. Изчислява и комбинира "spur" и "root" пътища. Управлява изключенията чрез exclusionCandidates.

2.3. Мрежова комуникация

Класовете Client и Server реализират базова TCP клиент-сървърна архитектура.

- Client – Клас за взаимодействие с потребителя. Изисква входни параметри: брой ребра, начален и краен връх, брой пътища, изпраща данни на сървъра и изчаква отговор. След това отпечатва получените пътища с тежест;
- Server – Клас, който обработва заявки от клиенти. Приема и анализира входа, използва Yen за изчисление на k най-кратки пътища и връща резултатен списък от пътища;
- ClientHandler — Вътрешен клас на Server: Обработва входни данни ред по ред, инициализира графа и върховете, изпълнява бизнес логиката и връща резултата.

3. Описание на алгоритъма на Йен

Алгоритъмът на Йен или още Yen's K-Shortest Paths Algorithm се използва за намиране на K най-кратки прости пътища без цикли между два върха в претеглен ориентиран граф. Той е подходящ за случаи, в които не е достатъчен само най-краткият път — например в маршрутизиране, транспорт, мрежови системи и др..

Генерират се всички K най-кратки пътища един по един, като след първия т.е. най-краткия, всяка следваща итерация изчислява алтернативен път чрез комбинация от предишни пътища и „отклонения“ (spur пътища) от тях.

3.1. Основни дефиниции

- Основен път (Root path) – Началната част от вече открит път до определен връх;
- Отклонение (Spur path) – Нов път от текущ връх (spur node) до целевия връх, избягвайки повторение на досега открити пътища;
- Общ път (Total path) – Комбинация от root path и spur path.

3.2. Стъпка по стъпка

Изчисляване на най-краткия път P1:

- Използва се алгоритъмът на Дейкстра без ограничения.
- Резултатът е първият и най-кратък път от източника до целта.

Инициализация:

- Списък paths съдържа вече откритите K пътища.
- Списък candidates събира временни кандидати за следващ най-кратък път.

Основен цикъл (до K пъти): За всяка итерация i от 1 до K-1:

- Намира се предишният най-кратък път Pi.
- За всяка възможна позиция j във Pi (с изключение на последния връх):
  - Определя се spur node – връх на позиция j.
  - Разделя се Pi на: Root path: от началото до j, и Spur path: от j до целта (ще се търси нов)
- Изключват се ребра, които биха дублирали вече открити пътища със същата начална част (root path).
- Алгоритъмът на Дейкстра изчислява нов spur path от spur node до целта, без вече използвани ребра от root path.
- Ако е намерен валиден spur path: Обединява се с root path и се добавя в списъка candidates, ако не е дубликат
- Избира се най-краткият път от candidates и го добавя към paths.

Край на алгоритъма:

- Алгоритъмът приключва, когато:
  - Са намерени K пътища, или
  - Няма повече валидни кандидати (графът е изчерпан)

3.3. Сложност

- В най-лошия случай сложността е $O(K \times N \times (E + N \log N))$, където K – брой търсени пътища, N – брой върхове, и E – брой ребра;
- Зависи силно от ефективността на алгоритъма на Дейкстра и структурата на графа.

4. Изграждане и използване

4.1. Изграждане

- За изграждане на приложението са нужни JDK и Maven. При разработката е използван JDK 22, и по-стари версии НЕ работят.

4.2. Използване

- Първо се стартира клас Server, като в него има статично поле за номер на порт, който може да бъде променян при нужда. По подразбиране е 1968;
- След това се стартират програми, изпълняващи Client класа. Ако портът на сървъра е променен, това трябва да бъде отразено и в класа за клиента. Във всяка от тях от потребителя се очаква да въведе:

- На първия ред: n — брой ребра в графа, начален връх, краен връх и k, където k е броят на най-къси пътища, които да бъдат намерени;
- На следващите n на брой редове: име на връх начало, име на връх край, тегло на реброто между съответните два върха.

- За разделители се използват интервали;
- След подаване на нужната информация, сървърът изпълнява алгоритъма на Йен и връща отговор:
- Ако няма път между двата върха се връща „EMPTY PATH“. В противен случай се връщат първите k най-къси пътища, подредени в списък. Всеки път е изразен чрез списък от върхове и теглото му.

Бележка:

- IntelliJ IDEA 2024.3

5. Примерни резултати

В тази секция са представени резултатите от модулни тестове, които проверяват коректността на имплементацията на два ключови алгоритъма – на Дейкстра и на Йен. Използвани са примерни графи със сравнително малък брой върхове и ребра, но с разнообразни маршрути и тегла, които позволяват ефективно тестване на функционалността.

5.1. Алгоритъм на Дейкстра

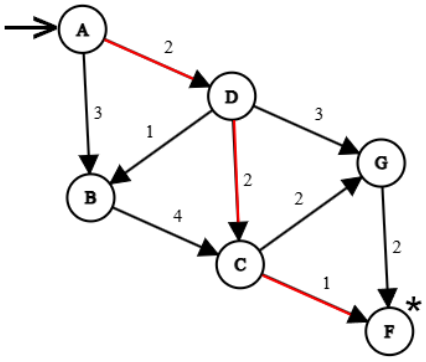
- В теста testDijkstraCalculatesShortestPath, се изчислява най-краткият път от връх A до връх F, при липса на изключени ребра. Конструкцията на графа включва няколко възможни маршрута с различни тегла.

Най-краткият път, намерен от алгоритъма:

A → D → C → F

Обща дължина: 5.0

Анализът показва, че този маршрут използва най-ефективните свързващи ребра по отношение на теглото: A → D (2.0), D → C (2.0), C → F (1.0)



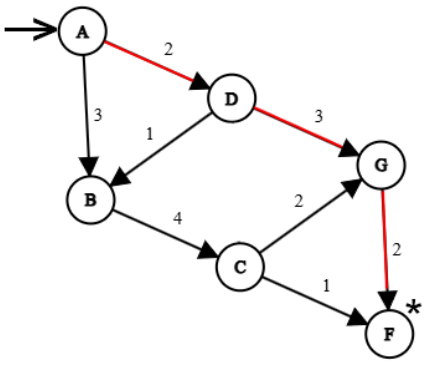
- Във втория тест testDijkstraCalculatesShortestPathWithExcludedEdge, реброто DC е изключено. Това принуждава алгоритъма да търси алтернативен маршрут.

Нов най-кратък път:

A → D → G → F

Обща дължина: 7.0

Изключването на ключово ребро (DC) води до по-дълъг път, но алгоритъмът все пак намира валидно решение, което потвърждава устойчивостта му при динамични ограничения.

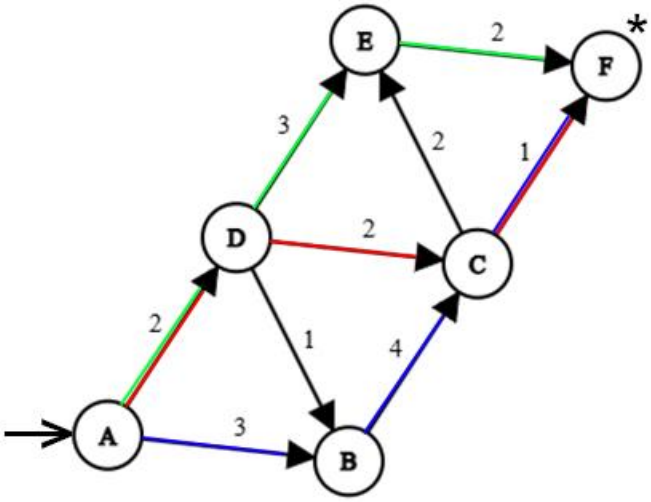


5.2 Резултати от алгоритъма на Йен

- В теста calculateKShortestPaths се търсят 3 най-кратки прости пътища от връх A до връх F. Графът съдържа множество свързаности и алтернативи между възлите. Получените пътища, подредени по нарастваща дължина, са:

1	**A → D → C → F**	**5.0**
2	**A → D → E → F**	**7.0**
3	**A → B → C → F**	**8.0**

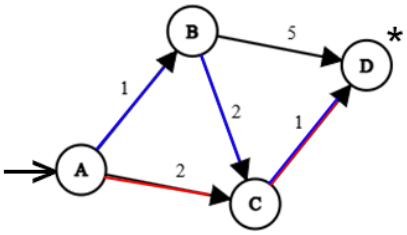
Първият път е същият като откритията от алгоритъма на Дейкстра, тъй като той винаги е най-краткият. Вторият и третият път са алтернативи, намерени чрез модифицирани версии на графа, където се изключват определени ребра или маршрути от предишни решения според логиката на Йен. Всеки следващ път е по-дълъг, но алгоритъмът успешно избягва дублиране на вече използвани маршрути и намира уникални решения.



5.3. Потребителски зададени входни данни

```
Enter NUM_OF_EDGES, START, TARGET, K (space-separated): 5 A D 2
Enter each edge in the format: FROM TO WEIGHT
Edge 1: A B 1
Edge 2: A C 2
Edge 3: B C 2
Edge 4: C D 1
Edge 5: B D 5

Result from server:
[[A, C, D] (3,00), [A, B, C, D] (4,00)]
```



6. Заключение

6.1. Обобщение

Проектът представя разработка, имплементация и тестване на два алгоритъма за намиране на най-кратки пътища в ориентиран и претеглен граф – алгоритъма на Дейкстра и алгоритъма на Йен за намиране на K най-кратки пътя. Чрез проведените експерименти и анализи се демонстрират приложимостта, гъвкавостта и точността на реализираните алгоритми в различни графови структури и сценарии.

Алгоритъмът на Дейкстра намира най-краткия път от даден начален връх до краен връх при положителни тегла на ребрата. Имплементацията му поддържа изключване на ребра, което я прави особено ефективна в динамична среда. Проведените тестове потвърждават коректността на алгоритъма при различни графови конфигурации – както с, така и без изключени ребра.

Алгоритъмът на Йен разширява възможностите за анализ, като изчислява K най-кратки прости пътища без цикли между два върха. Това осигурява надеждна алтернатива в случай на промяна в условията на мрежата. В примери с реалистични графови структури алгоритъмът намира всички релевантни маршрути, подредени по нарастваща дължина.

Сравнителният анализ между двата алгоритъма показва, че Дейкстра е подходящ при нужда от единствено оптимално решение в стабилна среда. Йен осигурява по-голяма устойчивост и адаптивност чрез намиране на алтернативни

маршрути, което го прави приложим в реални сценарии с динамично променящи се условия.

И двата алгоритъма преминават успешно поредица от юнит тестове, обхващащи както стандартни, така и гранични случаи. Получените резултати отговарят на теоретичните очаквания и доказват надеждността на имплементациите.

6.2. Използвана литература

- https://en.wikipedia.org/wiki/Yen%27s_algorithm
- https://haydushki-fmi/yen_algorithm
- https://docs.oracle.com/javase/tutorial/
- https://neo4j.com/docs/graph-data-science/current/algorithms/yens/
- https://people.csail.mit.edu/minilek/yen_kth_shortest.pdf